

LOAN PREDICTION SYSTEM

By

Patricia Stanley (MST03-0053)

Submitted to Scifor Technologies



Script. Sculpt. Socialize

UNDER THE GUIDENCE OF

Urooj Khan

TABLE OF CONTENTS

✓ Abstract	3
✓ Introduction	4
✓ Technology Used	5
✓ Dataset Information	6
✓ Methodology	7
✓ Model Deployment	8
✓ Code Snippet	9 - 15
✓ Results and Discussion	16
✓ Conclusion	17
✓ References	18

ABSTRACT:

The Loan Prediction System is a machine learning-based project aimed at predicting the approval status of loan applications. By analyzing applicant data such as income, credit history, and employment status, the system provides an accurate prediction of whether a loan application will be approved. The project utilizes various machine learning models, one of them includes the Support Vector Machines (SVM), which is used to enhance prediction accuracy. The system is deployed using Streamlit, offering a user-friendly interface for real-time predictions and data visualization. This report details the technologies used, dataset, methodology, code implementation, results, and conclusions.

INTRODUCTION:

Loan prediction and management are critical components of the financial industry. Financial institutions must predict the likelihood of a borrower defaulting on a loan to minimize financial risk. Furthermore, managing loans effectively is essential to ensure timely payments and minimize defaults. However, traditional methods of loan prediction and management rely on subjective decision-making, which can lead to errors and inefficiencies. In recent years, machine learning has emerged as a powerful tool to improve loan prediction accuracy and streamline loan management processes. In this research , we propose a loan prediction and management website that employs machine learning algorithms to improve loan prediction accuracy and optimize loan management processes.

The Loan Prediction System is a sophisticated and all-inclusive software solution crafted to optimize and streamline the entire loan lifecycle within financial institutions. In today's fast-paced financial environment, effective loan management is vital for banks, credit unions, and other lending organizations to maintain their competitive edge, mitigate risks, and ensure seamless operations. The Loan Prediction System incorporates cutting-edge technologies and data-driven processes to facilitate a smooth flow of loan origination, processing, approval, disbursement, and monitoring.

The ability to accurately predict the likelihood of loan approval helps in reducing the risk of default and optimizing the loan approval process. This project develops a Loan Prediction System that automates the prediction process using machine learning algorithms. The system analyzes various factors such as applicant income, credit history, and property area to predict the approval status of a loan application. The goal is to enhance decision-making efficiency and reduce human bias.

TECHNOLOGY USED:

The Loan Prediction System leverages the following technologies:

- **Python:** The primary programming language used for data processing, model training, and deployment.
- **Pandas and NumPy:** Libraries for data manipulation and analysis.
- **Scikit-learn:** A machine learning library used for implementing various models like SVM, Random Forest, and Logistic Regression.
- **Matplotlib and Seaborn:** Visualization libraries for plotting data distributions and model performance metrics.
- **Streamlit:** A Python framework used to deploy the machine learning model as a web application.



DATASET INFORMATION:

The dataset used for this project contains information about loan applicants, with the following features:

- **Loan_ID:** Unique identifier for each loan application.
- **Gender:** Gender of the applicant.
- **Married:** Marital status of the applicant.
- **Dependents:** Number of dependents.
- **Education:** Educational background of the applicant.
- **Self_Employed:** Whether the applicant is self-employed.
- **ApplicantIncome:** Monthly income of the applicant.
- **CoapplicantIncome:** Monthly income of the co-applicant.
- **LoanAmount:** The loan amount requested.
- **Loan_Amount_Term:** Term of the loan in months.
- **Credit_History:** Credit history status (1 indicates good credit history).
- **Property_Area:** The area where the property is located (Urban, Semi-Urban, or Rural).
- **Loan_Status:** The target variable indicating loan approval status (Y for approved, N for not approved).

METHODOLOGY:

The methodology for the Loan Prediction System involves the following steps:

1. Data Preprocessing

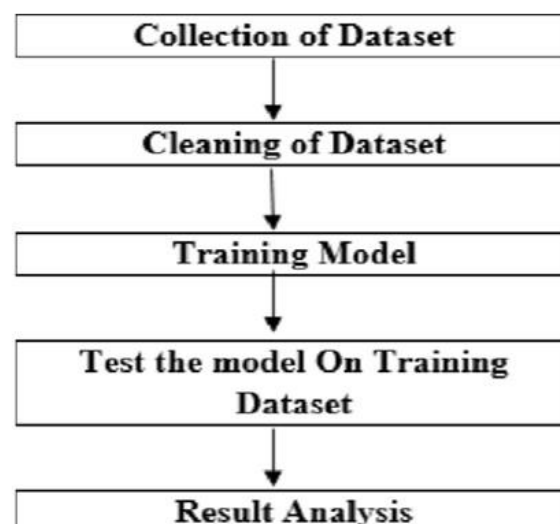
- **Handling Missing Values:** Missing data in columns such as LoanAmount and Credit_History is imputed using the median or mode.
- **Encoding Categorical Variables:** Categorical variables like Gender, Education, and Property_Area are converted into numerical formats using one-hot encoding.
- **Feature Scaling:** Features such as ApplicantIncome and LoanAmount are scaled using standardization to improve model performance.

2. Model Selection

- **Support Vector Machine (SVM):** Selected for its ability to handle high-dimensional data and provide accurate classification results.

3. Model Training and Evaluation

- The dataset is split into training and testing sets (80/20 split).
- The models are trained on the training data.
- The performance is evaluated using accuracy, precision, recall, and F1-score metrics.



1.1 FlowChart

MODEL DEPLOYMENT WITH STREAMLIT:

Streamlit Overview

Streamlit is an open-source Python framework that simplifies the process of creating and deploying machine learning web applications. It allows developers to turn Python scripts into interactive web apps with minimal effort, focusing on data-driven projects. In this project, here Streamlit is used to deploy the Loan Prediction System, enabling users to make predictions and visualize results in real-time.

Deployment Process

1. **Integration with Streamlit:** The trained machine learning model is integrated into a Streamlit script, which handles user inputs and displays prediction results.
2. **User Interface Design:** The Streamlit app features a simple, intuitive interface where users can input applicant details such as income, credit history, and property area.
3. **Real-Time Prediction:** Upon submitting the inputs, the app processes the data, applies the trained model, and returns a prediction (approved or not approved) almost instantaneously.
4. **Data Visualization:** The app also provides visual insights, such as the distribution of approved and rejected loans, allowing users to better understand the model's decisions.



Streamlit

CODE SNIPPET:

Implementation

```
: # Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pickle
```

```
: # Load the dataset
df = pd.read_csv('loan.csv')
```

```
# Displays the first 5 rows of the DataFrame.
df.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0

```
: # Displays the Last 5 rows of the DataFrame.
df.tail()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
609	LP002978	Female	No	0	Graduate	No	2900	0.0	71.0	360.0	1.0
610	LP002979	Male	Yes	3+	Graduate	No	4106	0.0	40.0	180.0	1.0
611	LP002983	Male	Yes	1	Graduate	No	8072	240.0	253.0	360.0	1.0
612	LP002984	Male	Yes	2	Graduate	No	7583	0.0	187.0	360.0	1.0
613	LP002990	Female	No	0	Graduate	Yes	4583	0.0	133.0	360.0	0.0

```
: # Display the shape of the DataFrame(prints the number of rows and columns)
df.shape
```

```
: (614, 13)
```

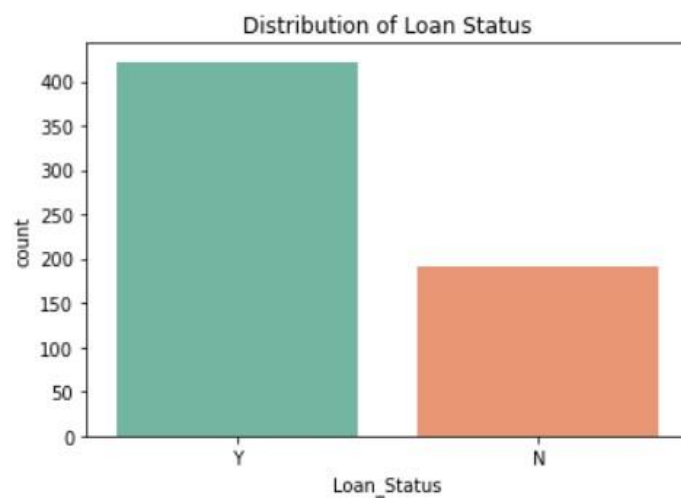
```
: # Provides an overview of the dataset, including data types, non-null counts, and memory usage.
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                614 non-null    object
1   Gender                 601 non-null    object
2   Married                611 non-null    object
3   Dependents             599 non-null    object
4   Education              614 non-null    object
5   Self_Employed          582 non-null    object
6   ApplicantIncome        614 non-null    int64
7   CoapplicantIncome      614 non-null    float64
8   LoanAmount             592 non-null    float64
9   Loan_Amount_Term       600 non-null    float64
10  Credit_History         564 non-null    float64
11  Property_Area          614 non-null    object
12  Loan_Status            614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

```
: # Displays the statistical summary of statistics for the numerical columns in the dataset, such as mean, standard deviation, min,
df.describe()
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.000000	564.000000
mean	5403.459283	1621.245798	146.412162	342.000000	0.842199
std	6109.041673	2926.248369	85.587325	65.12041	0.364878
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2877.500000	0.000000	100.000000	360.000000	1.000000
50%	3812.500000	1188.500000	128.000000	360.000000	1.000000
75%	5795.000000	2297.250000	168.000000	360.000000	1.000000
max	81000.000000	41667.000000	700.000000	480.000000	1.000000

```
: # Visualize the distribution of the target variable
plt.figure(figsize=(6, 4))
sns.countplot(x='Loan_Status', data=df, palette='Set2')
plt.title('Distribution of Loan Status')
plt.show()
```



```

: # Visualize the relationship between features using a pairplot
sns.pairplot(df, hue='Loan_Status', palette='Set1', diag_kind='kde')
plt.suptitle('Pairplot of Features Colored by Loan Status', y=1.02)
plt.show()

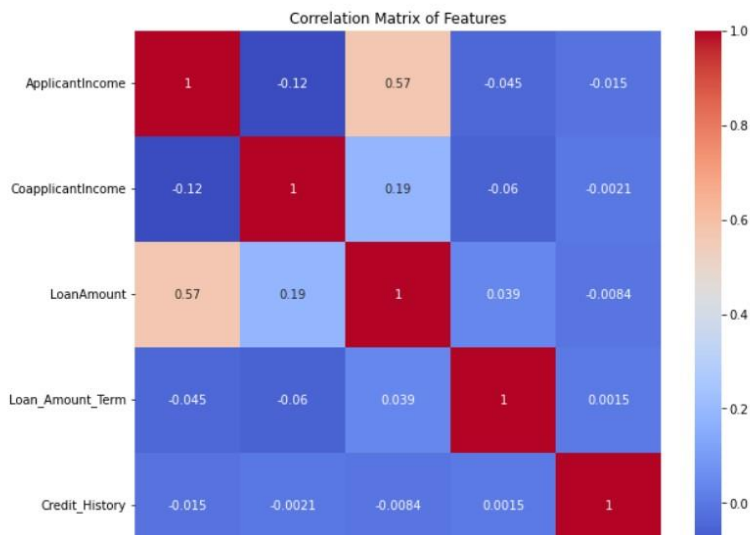
```



```

: # Visualize the correlation matrix
plt.figure(figsize=(10, 8))
corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', square=True)
plt.title('Correlation Matrix of Features')
plt.show()

```



Data Cleaning, Preprocessing and Model Building

```

# Checks for missing values in each column, displaying the count of null values.
df.isnull().sum()

```

```

Loan_ID          0
Gender           13
Married          3
Dependents       15
Education         0
Self_Employed    32
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount       22
Loan_Amount_Term 14
Credit_History   50
Property_Area     0
Loan_Status       0
dtype: int64

```

```
] : # Drops all rows that contain any missing values from the DataFrame.  
df.dropna(inplace=True) # Handling missing values
```

```
] : # Create and fit label encoders for categorical features  
encoders = {}  
for column in ['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'Property_Area']:  
    le = LabelEncoder()  
    df[column] = le.fit_transform(df[column])  
    encoders[column] = le
```

```
# Saves the fitted LabelEncoder objects to a file for later use or  
# Save encoders for use in the Streamlit app  
with open('label_encoders_svm.pkl', 'wb') as file:  
    pickle.dump(encoders, file)
```

```
# Separates the dataset into features (X) and target variable (y). The Loan_ID column is excluded as it's not needed for prediction  
X = df.drop(['Loan_ID', 'Loan_Status'], axis=1) # Exclude Loan_ID from features  
y = df['Loan_Status']
```

```
] : # Standardize the features  
scaler = StandardScaler()  
X = scaler.fit_transform(X)  
  
# Standardizes the features by scaling them to have a mean of 0 and a standard deviation of 1.
```

```
] : # Splits the data into training (70%) and testing (30%) sets.  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
] : # Initializes and trains an SVM classifier with a linear kernel on the training data.  
model.fit(X_train, y_train)
```

```
] : 

SVC

  
SVC(kernel='linear', probability=True, random_state=42)
```

Save the model

```
] : with open('loan_model_svm.pkl', 'wb') as file:  
    pickle.dump(model, file)
```

```
] : # Make predictions and evaluate  
y_pred = model.predict(X_test)
```

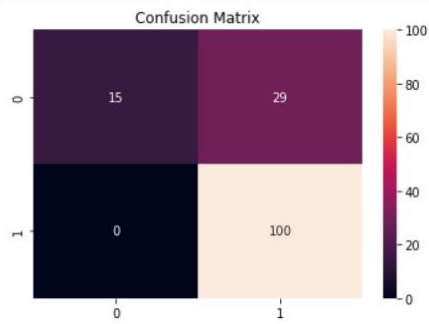
```
] : # Accuracy score  
accuracy = accuracy_score(y_test, y_pred)  
print(f'Accuracy: {accuracy:.2f}')
```

Accuracy: 0.80

```

: # Confusion matrix
# Plots the confusion matrix, showing the counts of true positives, true negatives, false positives, and false negatives.
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.title('Confusion Matrix')
plt.show()

```



```

: # Classification report (Displays a detailed classification report, including precision, recall, F1-score, and support for each class)
print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
N	1.00	0.34	0.51	44
Y	0.78	1.00	0.87	100
accuracy			0.80	144
macro avg	0.89	0.67	0.69	144
weighted avg	0.84	0.80	0.76	144

CODE SNIPPET FOR STREAMLIT DEPLOYMENT:

```
1 import streamlit as st
2 import pandas as pd
3 import pickle
4
5 # Load the trained model, encoders, and scaler
6 with open('loan_model_svm.pkl', 'rb') as file:
7     model = pickle.load(file)
8
9 with open('label_encoders_svm.pkl', 'rb') as file:
10     encoders = pickle.load(file)
11
12 with open('scaler_svm.pkl', 'rb') as file:
13     scaler = pickle.load(file)
14
15 # Define the Streamlit app
16 st.title('Loan Prediction System - SVM Model')
17
18 # Input fields for user data
19 gender = st.selectbox('Gender', ['Male', 'Female'])
20 married = st.selectbox('Married', ['Yes', 'No'])
21 dependents = st.selectbox('Dependents', ['0', '1', '2', '3+'])
22 education = st.selectbox('Education', ['Graduate', 'Not Graduate'])
23 self_employed = st.selectbox('Self Employed', ['Yes', 'No'])
24 applicant_income = st.number_input('Applicant Income', min_value=0)
25 coapplicant_income = st.number_input('Coapplicant Income', min_value=0)
26 loan_amount = st.number_input('Loan Amount', min_value=0)
27 loan_amount_term = st.number_input('Loan Amount Term', min_value=0)
28 credit_history = st.selectbox('Credit History', [1.0, 0.0])
29 property_area = st.selectbox('Property Area', ['Urban', 'Semiurban', 'Rural'])
30
31 # Preprocess user input using the same encoders and scaler as during training
32 inputs = pd.DataFrame({
33     'Gender': [gender],
34
35     'Married': [married],
36     'Dependents': [dependents],
37     'Education': [education],
38     'Self_Employed': [self_employed],
39     'ApplicantIncome': [applicant_income],
40     'CoapplicantIncome': [coapplicant_income],
41     'LoanAmount': [loan_amount],
42     'Loan_Amount_Term': [loan_amount_term],
43     'Credit_History': [credit_history],
44     'Property_Area': [property_area]
45 })
46
47 # Apply the encoders
48 for column in ['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'Property_Area']:
49     inputs[column] = encoders[column].transform(inputs[column])
50
51 # Apply scaling
52 inputs = scaler.transform(inputs)
53
54 # Predict using the model and show probabilities
55 if st.button('Predict'):
56     probability = model.predict_proba(inputs)[0][1] # Probability of class 1 (Loan Approved)
57
58     # Set a custom threshold for approval
59     threshold = 0.5 # You can adjust this value based on your needs
60
61     if probability > threshold:
62         st.write(f'Approval Probability: {probability:.2f}')
63         st.success('Loan Approved')
64     else:
65         st.write(f'Approval Probability: {probability:.2f}')
66         st.error('Loan Denied')
```

Output for the Streamlit Deployed App:

Share ☆ ↺ ⋮

Loan Prediction System - SVM Model

Gender
Male ▾

Married
Yes ▾

Dependents
0 ▾

Education
Graduate ▾

Self Employed
N ▾

⏪ Manage app

RESULTS AND DISCUSSION:

The SVM model achieved an accuracy of 80% on the test set, demonstrating strong performance in predicting loan approval status. The confusion matrix showed that most loan applications were classified correctly, with a few misclassifications.

The Streamlit deployment was successful, providing a responsive and interactive platform for users to make predictions based on applicant data. The visualizations offered by the app helped users understand the distribution and impact of various features on loan approval.

CONCLUSION:

The Loan Prediction System successfully automates the loan approval prediction process using machine learning. The SVM model, integrated with Streamlit, offers a robust and user-friendly solution for real-time loan status prediction. By deploying the system as a web application, financial institutions can easily access and utilize the model for decision-making.

Future work could involve expanding the dataset with additional features, integrating deep learning models, and enhancing the interpretability of the model's predictions.

REFERENCES:

- Scikit-learn Documentation: <https://scikit-learn.org/>
- Streamlit Documentation: <https://streamlit.io/>
- "Support Vector Machines: Theory and Applications," Springer:
<https://link.springer.com/>
- Dataset Source: Kaggle Loan Prediction Dataset.
Link: <https://www.kaggle.com/datasets/ninzaami/loan-predication>
- Loan Prediction Using Machine Learning Algorithms :
<https://medium.com/@sangitapokhrel911/loan-prediction-using-machine-learning-algorithms-96d17771e31>